

Appunti sul progetto di Pneumonia Detection

Contents

1	ResNet	2
1.1	Bottleneck	2
1.1.1	Batch Normalization	3
1.2	Struttura di ResNet-50	3
1.2.1	Global Average Pooling	3
2	Feature Pyramid Network	4
2.1	Motivazione	4
2.2	Struttura della FPN	4
2.3	FPN utilizzata nel progetto	5
3	Detection Head	5
3.1	Idea generale	5
3.2	RetinaNet	6
3.3	FCOS	6
3.4	Detection head del paper sulla pneumonia	7
3.5	Detection head del progetto	7
3.6	Differenze rispetto ai paper	7
3.6.1	Group Normalization	8

1 ResNet

La ResNet (Residual Network) introduce le *residual connections* per facilitare l'addestramento di reti molto profonde. L'idea è che gli strati convoluzionali non debbano necessariamente apprendere direttamente la funzione desiderata $H(x)$, ma possano apprendere una funzione residua $F(x)$ rispetto all'input:

$$F(x) = H(x) - x$$

Il risultato del blocco viene quindi ottenuto come:

$$H(x) = F(x) + x$$

L'input x viene quindi mantenuto attraverso una *shortcut connection* e sommato all'output della trasformazione residua. Questa struttura facilita l'ottimizzazione delle reti profonde, poiché, nel caso in cui la trasformazione ottimale fosse vicina all'identità, il blocco deve imparare una correzione rispetto all'input invece dell'intera trasformazione. Il paper originale introduce questa formulazione come principio fondamentale della residual learning.

1.1 Bottleneck

ResNet-50 utilizza un particolare tipo di residual block chiamato *bottleneck block*. Il blocco è costituito da tre convoluzioni:

$$1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$$

La prima convoluzione 1×1 riduce il numero di canali della rappresentazione. La convoluzione 3×3 , che è la principale convoluzione spaziale del blocco, opera quindi su un numero ridotto di canali, diminuendo il costo computazionale. Infine, la seconda convoluzione 1×1 aumenta nuovamente il numero di canali, riportandolo alla dimensione necessaria per effettuare la somma con la shortcut.

Ad esempio, un blocco può trasformare una rappresentazione:

$$256 \rightarrow 64 \rightarrow 64 \rightarrow 256$$

dove il passaggio a 64 canali costituisce il *bottleneck*.

La struttura completa del blocco può essere rappresentata come:

$$x \rightarrow 1 \times 1 \rightarrow BN \rightarrow ReLU \rightarrow 3 \times 3 \rightarrow BN \rightarrow ReLU \rightarrow 1 \times 1 \rightarrow BN$$

e, in parallelo, l'input viene trasmesso attraverso la shortcut. I due rami vengono poi sommati e viene applicata una ReLU finale:

$$y = ReLU(F(x) + x)$$

Quando le dimensioni dell'input e dell'output non coincidono, la shortcut viene trasformata tramite una convoluzione 1×1 per adattarne numero di canali e, se necessario, risoluzione spaziale.

Il bottleneck è stato introdotto nel paper originale proprio per rendere più efficiente la costruzione di reti molto profonde: le convoluzioni 1×1 riducono e poi aumentano le dimensioni, lasciando alla convoluzione 3×3 una rappresentazione più piccola su cui operare.

1.1.1 Batch Normalization

La *Batch Normalization* (BN) normalizza le attivazioni prodotte da una convoluzione utilizzando la media e la varianza calcolate sul mini-batch. Successivamente, le attivazioni normalizzate vengono trasformate tramite due parametri apprendibili, che permettono alla rete di adattarne nuovamente scala e traslazione.

In ResNet, la Batch Normalization viene applicata dopo ogni convoluzione e prima della funzione di attivazione ReLU. La sua funzione è rendere più stabile l'addestramento della rete e facilitare l'ottimizzazione dei numerosi strati convoluzionali.

1.2 Struttura di ResNet-50

ResNet-50 è costruita come una sequenza di quattro stage principali, ciascuno composto da più Bottleneck block. La configurazione di ResNet-50 utilizza rispettivamente:

$$[3, 4, 6, 3]$$

Bottleneck nei quattro stage `conv2_x`, `conv3_x`, `conv4_x` e `conv5_x`.

La struttura può essere riassunta come:

$$\text{Input} \rightarrow 7 \times 7 \text{ Conv} \rightarrow \text{MaxPool} \rightarrow \text{conv2_x} \rightarrow \text{conv3_x} \rightarrow \text{conv4_x} \rightarrow \text{conv5_x}$$

Per un input RGB di dimensione 224×224 , le principali dimensioni delle feature map sono:

$$224 \times 224 \rightarrow 112 \times 112 \rightarrow 56 \times 56 \rightarrow 28 \times 28 \rightarrow 14 \times 14 \rightarrow 7 \times 7$$

mentre il numero di canali aumenta progressivamente:

$$64 \rightarrow 256 \rightarrow 512 \rightarrow 1024 \rightarrow 2048.$$

Il primo layer utilizza una convoluzione 7×7 con stride 2, seguita da un max pooling con stride 2. I successivi cambiamenti di risoluzione avvengono all'inizio degli stage `conv3_x`, `conv4_x` e `conv5_x`, dove il primo Bottleneck utilizza stride 2. Nel codice, quindi, il primo Bottleneck di ogni nuovo stage è anche quello che adatta la dimensione della shortcut quando necessario.

1.2.1 Global Average Pooling

Al termine della rete, la ResNet originale utilizza un *Global Average Pooling* (GAP) per trasformare la feature map finale in un vettore compatto.

Nel caso di ResNet-50, la feature map finale ha dimensione:

$$7 \times 7 \times 2048.$$

Il Global Average Pooling calcola, per ogni canale, la media dei valori presenti su tutte le posizioni spaziali:

$$y_c = \frac{1}{HW} \sum_{i=1}^H \sum_{j=1}^W x_{c,i,j}.$$

La dimensione spaziale viene quindi ridotta da 7×7 a 1×1 , ottenendo:

$$7 \times 7 \times 2048 \rightarrow 1 \times 1 \times 2048.$$

Successivamente il tensore viene appiattito e passato al layer fully connected che produce le predizioni finali.

Il vantaggio del Global Average Pooling è che permette di riassumere l'intera mappa spaziale di ciascun canale in un singolo valore, evitando l'utilizzo di una grande quantità di parametri tipica di un fully connected applicato direttamente alla feature map. Il GAP non verrà usato nel caso in cui ResNet debba funzionare come backbone.

2 Feature Pyramid Network

2.1 Motivazione

Una rete convoluzionale profonda, come ResNet-50, produce una gerarchia di feature map a diverse risoluzioni spaziali. I layer più profondi hanno una risoluzione spaziale più bassa, ma contengono informazioni semantiche più ricche, mentre i layer più superficiali mantengono una maggiore risoluzione spaziale e una localizzazione più precisa, ma hanno informazioni semantiche meno forti.

Questo rappresenta un problema per l'object detection, perché gli oggetti possono avere dimensioni molto diverse. Le feature profonde sono utili per riconoscere *cosa* è presente nell'immagine, ma sono meno precise nel dire *dove* si trova. Al contrario, le feature superficiali sono più precise spazialmente, ma hanno una rappresentazione semantica più debole.

La Feature Pyramid Network (FPN) serve quindi a combinare queste due informazioni. L'idea è prendere le feature semantiche dei layer profondi e propagarle verso i layer a maggiore risoluzione, combinandole con le feature superficiali. In questo modo si ottengono feature map a diverse scale che mantengono sia una buona informazione semantica sia una buona precisione spaziale.¹

2.2 Struttura della FPN

La FPN originale utilizza le feature ottenute dagli ultimi blocchi residuali di ResNet, indicate come C_2 , C_3 , C_4 e C_5 . Queste feature hanno rispettivamente stride 4, 8, 16 e 32 rispetto all'immagine di input.

La costruzione della piramide parte dalla feature più profonda C_5 . Questa viene trasformata tramite una convoluzione 1×1 e costituisce il punto di partenza del percorso *top-down*. La feature viene poi sovracampionata di un fattore 2 e sommata alla corrispondente feature proveniente dal backbone, dopo che anch'essa è stata trasformata tramite una convoluzione 1×1 .

Il procedimento viene ripetuto progressivamente verso i livelli a maggiore risoluzione:

$$C_5 \rightarrow P_5$$

$$C_4 + \text{Upsample}(P_5) \rightarrow P_4$$

$$C_3 + \text{Upsample}(P_4) \rightarrow P_3$$

$$C_2 + \text{Upsample}(P_3) \rightarrow P_2$$

La convoluzione 1×1 viene utilizzata per portare le feature del backbone allo stesso numero di canali, mentre l'upsampling permette di allineare la risoluzione spaziale delle feature prima della somma.

Dopo la fusione viene applicata una convoluzione 3×3 per ottenere la feature finale di ciascun livello. Nel paper originale tutti i livelli della piramide hanno 256 canali.²

¹Tsung-Yi Lin et al., *Feature Pyramid Networks for Object Detection*, CVPR 2017.

²La FPN originale utilizza il percorso *top-down* con *lateral connections* e produce i livelli P_2 - P_5 .

2.3 FPN utilizzata nel progetto

Nel nostro progetto la FPN è leggermente diversa da quella originale, perché è stata adattata al detector utilizzato successivamente.

La ResNet-50 fornisce le feature:

$$C_3, C_4, C_5$$

con stride rispettivamente:

$$8, 16, 32.$$

A differenza della FPN originale, C_2 non viene utilizzata. La costruzione del percorso top-down rimane però la stessa:

$$C_5 \rightarrow P_5$$

$$C_4 + \text{Upsample}(P_5) \rightarrow P_4$$

$$C_3 + \text{Upsample}(P_4) \rightarrow P_3.$$

Anche in questo caso le feature provenienti dal backbone vengono prima proiettate tramite convoluzioni 1×1 a 256 canali. Dopo la somma viene applicata una convoluzione 3×3 .

La differenza principale è che il nostro modello non si ferma a P_3 , P_4 e P_5 , ma estende ulteriormente la piramide:

$$P_5 \rightarrow P_6 \rightarrow P_7.$$

P_6 viene ottenuta tramite una convoluzione 3×3 con stride 2 applicata a P_5 , mentre P_7 viene ottenuta nello stesso modo partendo da P_6 .

La piramide finale è quindi:

$$P_3, P_4, P_5, P_6, P_7$$

con stride:

$$8, 16, 32, 64, 128.$$

Questa scelta segue la struttura del detector descritto nel paper sulla pneumonia, che utilizza cinque livelli della feature pyramid con stride 8, 16, 32, 64 e 128.³

In sintesi, la FPN permette di avere feature a più scale, combinando informazioni semantiche profonde con informazioni spaziali più precise dei livelli superficiali. Nel nostro caso, questi livelli vengono poi utilizzati dal detector FCOS per effettuare le predizioni a scale differenti.

3 Detection Head

3.1 Idea generale

La *detection head* è il componente della rete che riceve le feature estratte dal backbone e dalla FPN e le trasforma nelle predizioni necessarie per l'object detection. In generale, una head può essere suddivisa in più branch, ognuna specializzata in un compito differente, ad esempio la classificazione dell'oggetto e la regressione della sua bounding box.

³Linghua Wu et al., *Pneumonia detection based on RSNA dataset and anchor-free deep learning detector*, Scientific Reports, 2024.

L'idea generale è quindi:

$$\text{Feature Map} \rightarrow \text{Detection Head} \rightarrow \begin{cases} \text{Classification} \\ \text{Localization} \end{cases}$$

Nel caso dei detector anchor-free, la predizione viene effettuata direttamente su ogni posizione spaziale della feature map invece di utilizzare bounding box predefinite come riferimento.

3.2 RetinaNet

Un esempio importante è RetinaNet. La rete utilizza due subnet separate, una per la classificazione e una per la regressione delle bounding box. Entrambe ricevono una feature map della FPN e sono costituite da più convoluzioni 3×3 , seguite da un layer finale di predizione.

La classification subnet produce, per ogni posizione spaziale, le probabilità relative alle classi e agli anchor presenti in quella posizione, mentre la box regression subnet produce i parametri necessari per correggere le coordinate degli anchor.⁴

La struttura può essere schematizzata come:

$$\text{FPN feature} \rightarrow \begin{cases} \text{Classification subnet} \\ \text{Regression subnet} \end{cases}$$

In RetinaNet le due subnet hanno la stessa struttura generale, ma utilizzano parametri separati. La classification subnet termina con KA output per posizione, mentre la regression subnet termina con $4A$ output, dove K è il numero di classi e A il numero di anchor per posizione.

3.3 FCOS

FCOS mantiene la separazione tra classificazione e regressione, ma elimina il riferimento agli anchor. Ogni posizione della feature map viene trattata come una possibile location dell'oggetto e la regressione predice direttamente le distanze dalla posizione ai quattro lati della bounding box:

$$(l, t, r, b).$$

La detection head di FCOS contiene quindi una branch per la classificazione e una branch per la regressione. Il paper descrive per entrambe una sequenza di convolutional layers applicata alle feature della FPN. In aggiunta viene introdotta una terza branch, la *center-ness branch*, che predice quanto una location sia vicina al centro dell'oggetto e serve a ridurre l'influenza delle predizioni di bassa qualità.⁵

Lo schema concettuale è quindi:

$$\text{FPN feature} \rightarrow \begin{cases} \text{Classification} \\ \text{Regression} \\ \text{Center-ness} \end{cases}$$

La classificazione determina *che cosa* è presente in una determinata location, la regressione determina *dove* si trova la bounding box, mentre la center-ness fornisce un'indicazione della qualità della location utilizzata per effettuare la predizione.

⁴Tsung-Yi Lin et al., *Focal Loss for Dense Object Detection*, ICCV 2017.

⁵Zhi Tian et al., *FCOS: Fully Convolutional One-Stage Object Detection*, ICCV 2019.

3.4 Detection head del paper sulla pneumonia

Il paper utilizzato come riferimento per questo progetto descrive una head più semplice. Ogni livello della feature pyramid viene elaborato inizialmente da una convoluzione 3×3 e successivamente da due convoluzioni 1×1 che producono due predizioni distinte, indicate come *CENTER* e *SCALE*.

$$\text{FPN feature} \rightarrow 3 \times 3 \rightarrow \begin{cases} 1 \times 1 \rightarrow \text{CENTER} \\ 1 \times 1 \rightarrow \text{SCALE} \end{cases}$$

Il branch CENTER viene utilizzato per individuare le location associate al centro degli oggetti, mentre il branch SCALE viene utilizzato per predire la scala della bounding box.⁶

3.5 Detection head del progetto

La detection head utilizzata nel progetto è più vicina a quella di FCOS che a quella descritta nel paper sulla pneumonia.

Ogni livello della FPN $P_3, \dots, P_7$ viene passato a una *DetectionHead*. La head è divisa in due tower principali:

$$\text{Feature} \rightarrow \begin{cases} \text{Classification tower} \\ \text{Regression tower} \end{cases}$$

La classification tower è costituita, nel progetto, da:

$$3 \times 3 \text{ Conv} \rightarrow \text{GroupNorm} \rightarrow \text{ReLU} \rightarrow 1 \times 1 \text{ Conv}.$$

La regression tower utilizza una struttura analoga:

$$3 \times 3 \text{ Conv} \rightarrow \text{GroupNorm} \rightarrow \text{ReLU}.$$

Dalla regression tower vengono poi prodotte due predizioni separate:

$$\text{Regression tower} \rightarrow \begin{cases} 1 \times 1 \rightarrow (l, t, r, b) \\ 1 \times 1 \rightarrow \text{centerness} \end{cases}$$

Pertanto, per ogni location di ogni livello della FPN, il modello produce:

$$\text{classification}, \quad (l, t, r, b), \quad \text{centerness}.$$

Nel progetto la classificazione produce attualmente un solo valore per location, perché viene utilizzata una sola classe target, la pneumonia.

3.6 Differenze rispetto ai paper

La struttura del progetto non coincide esattamente con nessuno dei due paper.

Rispetto al paper sulla pneumonia, il progetto aggiunge una *classification branch* esplicita e utilizza una regressione *LTRB*, più vicina alla formulazione FCOS, invece della distinzione CENTER/SCALE descritta nel paper.

Rispetto a FCOS, la struttura è semplificata: il paper originale utilizza quattro convolutional layers nelle branch di classificazione e regressione, mentre il progetto utilizza una sola convoluzione 3×3 per ciascuna tower, seguita da Group Normalization e ReLU.

⁶Linghua Wu et al., *Pneumonia detection based on RSNA dataset and anchor-free deep learning detector*, Scientific Reports, 2024.

Inoltre, FCOS condivide le head tra i diversi livelli della feature pyramid, mentre nel progetto vengono create cinque istanze separate di `DetectionHead`, una per ciascun livello $P_3, \dots, P_7$. Il paper FCOS descrive infatti la condivisione delle head tra i diversi livelli, pur introducendo modifiche specifiche per gestire i differenti range di scala.

Nel complesso, quindi, la detection head del progetto può essere considerata una *FCOS-like detection head*: ne mantiene la formulazione anchor-free, la classificazione per location, la regressione *LTRB* e la center-ness branch, ma utilizza una struttura più semplice e alcune scelte architetturali specifiche del progetto.

3.6.1 Group Normalization

La *Group Normalization* (GN) è una tecnica di normalizzazione delle feature map che divide i canali in gruppi e normalizza i valori all'interno di ciascun gruppo.

A differenza della Batch Normalization, la Group Normalization non utilizza le statistiche dell'intero batch. Questo la rende meno dipendente dalla dimensione del batch e quindi particolarmente utile quando si lavora con batch molto piccoli, situazione comune nell'object detection a causa dell'elevato costo computazionale delle immagini.

Nel progetto la Group Normalization viene utilizzata all'interno delle classification e regression tower della detection head:

$$\text{Conv} \rightarrow \text{GroupNorm} \rightarrow \text{ReLU}.$$

Il suo scopo è stabilizzare le attivazioni e rendere l'addestramento più robusto, senza richiedere batch grandi.